COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures & Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/16/26	**DATE SUBMITTED:**	10/16/26
NAME:	Arididdon, Gabriel Dwyane D. Esguerra, Paula G.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Hands-on Activity 13.1 Parallel Algorithms and Multithreading

1. PROCEDURE OUTPUT

LO A & ILO B:

Part 1: Simple One-Threaded Example

Output

```
Printing integer: 10
Printing string: T.I.P.
```

Analysis

The join() command waited for the printing of integers and printing the string to finish first before executing the main. Because without using join function, the main thread would finish the task early, the function of t1 will not execute, resulting it wouldn't print at all, resulting in an error.

Part 2: Multi-Threaded Example

Output

```
1 : Competent2 : Computer3 : Engineers0 : T.I.P.
```

Analysis

Based on the output, the threads executed concurrently that is why they are in the same line and overlappingly. The outcome of the code will execute all at the same time, it can be in different places, and different set, if they are printed multiple times. The execution time is faster than the regular execution time if multi-threaded isn't used.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions.

Questions

1. **Write a definition of multithreading and its advantages/disadvantages.**

   Aridiodon

   One advantage is improved responsiveness, allowing applications to continue working while other tasks run in the background. It also improves CPU utilization and may increase performance by allowing tasks to run in parallel. However, multithreading can make programs more complex and may cause synchronization issues, race conditions, or deadlocks when threads compete for shared resources.

   Esguerra

   The main function of multi-threading is to simultaneously run or execute multiple tasks. The common advantages of multi-threading is it can enhance performance by its execution time, it can also be better use of cache storage in utilization of resources and cpu resources. While the common disadvantages are it can have complex debugging and testing processes, overhead switching of context, and unpredictable results.

2. **Rationalize the use of multithreading by providing at least 3 use-cases.**

   Aridiodon

   Multithreading is useful in applications that need to handle multiple tasks efficiently. In web browsers, it allows activities such as loading pages, playing videos, and responding to user input to happen concurrently. In online games, separate threads can handle sound, user input, and background processing to improve responsiveness. In web servers, multithreading allows multiple client requests to be handled concurrently, improving the application's ability to serve multiple users.

   Esguerra

   Multi-threading increases responsiveness in interactive applications, if one thread is blockers, other threads can still continue executing. Second, threads share the memory and resources of their parent process by default, like message passing and shared memory. And third, creating and managing threads, we can avoid the need to separate allocations.

3. **Differentiate between parallelism and concurrency**

   Aridiodon

   Concurrency is the ability to manage multiple tasks that make progress during overlapping periods, even if they take turns executing on a single CPU core. Parallelism, on the other hand, involves executing multiple tasks simultaneously, typically using multiple CPU cores. The main goal of concurrency is to improve responsiveness and manage multiple tasks efficiently, while parallelism aims to increase processing speed. For example, switching between downloading a file and responding to

user input demonstrates concurrency, while processing different parts of an image simultaneously on separate CPU cores demonstrates parallelism.

Esguerra
Concurrency can manage multiple tasks to make processes during overlapping time periods. They can be run simultaneously on multiple processors. Parallelism can execute multiple tasks simultaneously using multiple processing units. It improves during the work across the processor's throughout and its computational speed in dividing work.

Part B: Create C++ Code and show a solution that satisfies the given requirements below

Code

```
#include <chrono>
#include <iostream>
#include <mutex>
#include <thread>

int total = 0;
std::mutex mtx;

void add(int n)
{

    std::this_thread::sleep_for(std::chrono::milliseconds(n * 10));
    std::lock_guard<std::mutex> lock(mtx);
    total += n;
}

int show()
{
    std::lock_guard<std::mutex> lock(mtx);
    return total;
}

int main()
{
    std::thread t1(add, 10);
```

```
    std::thread t2(add, 20);
    std::thread t3(add, 30);

    std::cout << "Display             : " << show() << std::endl;
    t1.join();
    std::cout << "After T1.join()     : " << show() << std::endl;
    t2.join();
    std::cout << "After T2.join()     : " << show() << std::endl;
    t3.join();
    std::cout << "After T3.join()     : " << show() << std::endl;
    return 0;
}
```

Output

```
Initial check (Threads running): 0
After T1.join(): 10
After T2.join(): 30
After T3.join(): 60
Final value (All joins complete): 60
```

Analysis

Aridido n	Esguerra

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

Code

```
#include <iostream>
#include <vector>
#include <thread>
```

```cpp
// Function to combine two sorted halves of an array
void merge(std::vector<int>& arr, int left, int mid, int right) {
    std::vector<int> temp(right - left + 1);
    int i = left, j = mid + 1, k = 0;

    // Compare elements and sort them into the temporary vector
    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) temp[k++] = arr[i++];
        else temp[k++] = arr[j++];
    }

    // Copy any remaining elements from the left half
    while (i <= mid) temp[k++] = arr[i++];

    // Copy any remaining elements from the right half
    while (j <= right) temp[k++] = arr[j++];

    // Copy the sorted elements back into the original array
    for (i = left, k = 0; i <= right; ++i, ++k) {
        arr[i] = temp[k];
    }
}

// Standard single-threaded sequential Merge Sort
void sequentialMergeSort(std::vector<int>& arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;

    sequentialMergeSort(arr, left, mid);
    sequentialMergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

// Multi-threaded Merge Sort utilizing Lambda Functions
void parallelMergeSort(std::vector<int>& arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;

    // Launch a thread using a lambda expression to handle the left half.
    // '[&]' captures local variables (arr, left, mid) safely by reference.
    std::thread leftThread([&]() {
```

```
        sequentialMergeSort(arr, left, mid);
    });

    // Launch a second thread using a lambda expression to handle the right half.
    std::thread rightThread([&]() {
        sequentialMergeSort(arr, mid + 1, right);
    });

    // Block the main execution thread until both worker threads complete their execution
    leftThread.join();
    rightThread.join();

    // Merge the two independent, fully sorted halves back together
    merge(arr, left, mid, right);
}

int main() {
    // Sample unsorted array
    std::vector<int> data = {38, 27, 43, 3, 9, 82, 10, 19, 50, 4, 1, 99};

    std::cout << "Original Array: ";
    for (int x : data) std::cout << x << " ";
    std::cout << "\n";

    // Run the parallel sorting algorithm
    parallelMergeSort(data, 0, data.size() - 1);

    std::cout << "Sorted Array:   ";
    for (int x : data) std::cout << x << " ";
    std::cout << "\n";

    return 0;
}
```

Output

```
Original Array: 38 27 43 3 9 82 10 19 50 4 1 99
Sorted Array:   1 3 4 9 10 19 27 38 43 50 82 99
```

Analysis

Aridon	Esguerra

Conclusion	
Aridon	**Esguerra**

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, explaining concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **We used Gemini to explain hard concepts and also with some code assistance in supplementary activity**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.